

This test plan outlines the testing strategy for the FRACA SERVCOM Inventory Management System, covering unit tests for stock and purchase logic, feature tests for all API endpoints, and manual acceptance tests for role-based access control and core CRUD operations. Automated tests run via PHPUnit with GitHub Actions CI/CD, ensuring system reliability and proper access enforcement before deployment.

INVENTORY MANAGEMENT SYSTEM

PHILIP LEE 22/07341

TEST PLAN

TEST PLAN

FRACA SERVCOM Inventory Management System

Document Version: 1.0

Date: March 2026

System: FRACA SERVCOM Inventory Management System

Technology Stack: Laravel 11, PHP 8.2, MySQL, Bootstrap 5, Vanilla JavaScript (ES6+), Vite

1. Introduction

1.1 Purpose

This test plan defines the testing strategy, scope, objectives, and procedures for the FRACA SERVCOM Inventory Management System. It covers unit testing, integration testing, feature (API endpoint) testing, and manual acceptance testing across all system modules.

1.2 Scope

- Testing covers:
- Backend API endpoints (authentication, CRUD operations, reports)
- Business logic (stock management, purchase/sale processing)
- Role-based access control (Admin, Manager, Staff)
- Frontend JavaScript modules (products, sales, purchases, categories, etc.)
- Responsive design across device breakpoints

1.3 Test Objectives

- Verify all API endpoints return correct responses and status codes
- Confirm role-based access control is enforced at both UI and API layers
- Validate stock management logic (increment, decrement, oversell prevention)
- Ensure data integrity across purchase and sale transactions
- Confirm form validation errors display correctly inline
- Verify the admin verification workflow for Manager delete operations

2. Test Environment

Component	Details
Backend	Laravel 11, PHP 8.2
Database (Testing)	SQLite in-memory (:memory:)
Database (Production)	MySQL — fraca_inventory
Frontend	Bootstrap 5, Vanilla JS, Vite
Test Framework	PHPUnit 11 (via Laravel)
HTTP Testing	Laravel TestCase, actingAs(), getJson(), postJson()
CI/CD	GitHub Actions (.github/workflows/test-and-newman.yml)

2.1 Application Access

Value	Item
Local URL	http://localhost:8000
App Name	FRACA SERVCOM Inventory Management System
Database	MySQL — fraca_inventory
DB Host	127.0.0.1:3306
DB Username	root (default XAMPP/local)
	DB Username
DB Password	(empty by default)

2.2 Test Users (Seeded)

Email	Password	Role	Access Level
admin@inventory.com	password123	Admin	Full access — all CRUD, user management, hard deletes
manager@inventory.com	password123	Manager	Full access except hard deletes; deletes require admin verification
staff@inventory.com	password123	Staff	staff@inventory.com

password123

3. Test Categories

3.1 Unit Tests (tests/Unit/)

StockServiceTest

Test ID	Test Name	Description	Expected Result
UT-001	test_stock_incremented_after_purchase	Stock increases by purchased quantity	current_stock = initial + purchased
UT-002	test_stock_decremented_after_sale	Stock decreases by sold quantity	current_stock = initial - sold
UT-003	test_overselling_prevented	Cannot sell more than available stock	current_stock < requested quantity is detectable
UT-004	test_manual_stock_adjustment_logged	Manual adjustment updates stock correctly	New stock value persisted
UT-005	test_low_stock_alert_generated	Alert condition detected when stock	current_stock < reorder_level is true

		$\leq$ reorder level	
UT-006	test_no_low_stock_alert_when_stock_sufficient	No alert when stock is above threshold	current_stock > reorder_level
UT-007	test_stock_history_polymorphic_tracking	StockHistory relationship accessible from Product	stockHistories() count $\geq$ 0
UT-008	test_purchase_transaction_atomicity	Stock unchanged if no operation performed	Stock equals initial value
UT-009	test_stock_never_goes_negative	Stock value is always $\geq$ 0	current_stock $\geq$ 0

3.2 PurchaseServiceTest

Test ID	Test Name	Description	Expected Result
UT-010	test_stock_incremented_after_purchase	Purchase increments product stock	current_stock = 10 + 5 = 15
UT-011	test_purchase_total_calculated_correctly	Total = sum of (qty × price) for all items	(5×10) + (3×20) = 110
UT-012	test_purchase_created_with_valid_data	Purchase record persisted to database	assertDatabaseHas('purchases')
UT-013	test_purchase_order_number_auto_generated	Each purchase gets a unique PO number	purchase_number is not null
UT-014	test_purchase_relationships	Purchase has supplier, user, and items	Relationships load correctly
UT-015	test_purchase_with_multiple_products	Multiple line items update all stocks	Each product stock incremented correctly
UT-016	test_purchase_status_transitions	Status can change from pending to completed	status = 'completed' after update
UT-017	test_purchase_cancellation_transaction_safety	Cancelled purchase does not corrupt stock	Stock unchanged

UT-018	test_purchase_transaction_creates_all_records	Purchase + items persisted correctly	Records successfully created
--------	---	--------------------------------------	------------------------------

Run command:
php artisan test tests/Unit/

3.2 Feature Tests(tests/Feature)

Feature tests verify HTTP endpoints end-to-end using Laravel's test client.

ApiEndpointVerificationTest

Test ID	Test Name	Endpoint	Expected Status
FT-001	test_login_endpoint_works	POST /api/login	200 + user + token
FT-002	test_products_endpoint_requires_authentication	GET /api/products	401 (unauthenticated)
FT-003	test_products_endpoint_works_with_authentication	GET /api/products	200 + data structure
FT-004	test_suppliers_endpoint_works	GET /api/suppliers	200
FT-005	test_customers_endpoint_works	GET /api/customers	200
FT-006	test_purchases_endpoint_works	GET /api/purchases	200
FT-007	test_sales_endpoint_works	GET /api/sales	200
FT-008	test_stock_adjustments_endpoint_works	GET /api/stock-adjustments	200

FT-009	test_dashboard_report_endpoint_works	GET /api/reports/dashboard	200
FT-010	test_sales_report_endpoint_works	GET /api/reports/sales	200
FT-011	test_purchases_report_endpoint_works	GET /api/reports/purchases	200
FT-012	test_stock_levels_report_endpoint_works	GET /api/reports/stock-levels	200
FT-013	test_inventory_valuation_report_endpoint_works	GET /api/reports/inventory-valuation	200
FT-014	test_logout_endpoint_works	POST /api/logout	200 + success: true
FT-015	test_csrf_token_is_available_in_web_routes	GET /	302 redirect to login

Run command:

php artisan test/Feature/

3.3 Manual Acceptance Tests

These tests will be performed manually in the browser.

Authentication

Test ID	Scenario	Steps	Expected Result	Test ID
AT-001	Admin login	Navigate to /login, enter admin@inventory.com / password123	Redirected to dashboard	AT-001
AT-002	Invalid login	Enter wrong password	Error message displayed	AT-002
AT-003	Logout	Click username → Logout	Redirected to login page	AT-003

Role-Based Access Control

Test ID	Scenario	Role	Expected Result	Test ID	Scenario
AT-005	Admin sees all nav items	Admin	Dashboard, Products, Sales, Purchases, Stock, Categories, Customers, Suppliers, Reports, Users	AT-005	Admin sees all nav items
AT-006	Staff sees limited nav	Staff	Dashboard, Products, Sales, Purchases only	AT-006	Staff sees limited nav
AT-007	Admin sees delete buttons	Admin	Trash icon visible in all tables	AT-007	Admin sees delete buttons

Products CRUD

Test ID	Scenario	Steps	Expected Result	Test ID	Scenario	Steps
AT-010	Add product	Click Add Product → fill form → Save	Product appears in table, success toast	AT-010	Add product	Click Add Product → fill form → Save
AT-011	Edit product	Click pencil icon → modify → Save	Updated data in table, success toast	AT-011	Edit product	Click pencil icon → modify → Save
AT-012	Delete product (Admin)	Click trash → confirm	Product removed, success toast	AT-012	Delete product (Admin)	Click trash → confirm

Sales and purchases

Test ID	Scenario	Steps	Expected Result	Test ID
AT-016	Create new sale	New Sale → add items → Save	Sale appears in table with KSh total	AT-016
AT-017	View sale details	Click invoice number	Modal shows line items and total	AT-017
AT-018	Create new purchase	New Purchase → add items → Save	Purchase appears in table	AT-018

Reports

Test ID	Scenario	Steps	Expected Result
AT-020	Generate inventory valuation	Reports → Inventory Valuation → Generate	Table with KSh values
AT-021	Export CSV	Generate report → Export to CSV	File downloads
AT-022	Date filter on sales report	Set date range → Generate	Filtered results displayed correctly

4. Test Execution

4.1 Running All Automated Tests

bash

php artisan test

4.2 Running with Coverage

bash

php artisan test --coverage

4.3 CI/CD Pipeline

- Tests run automatically on every push to main via GitHub Actions. The workflow
- Sets up PHP 8.2 environment
- Installs Composer dependencies
- Configures. env. testing with SQLite: memory: database
- Runs php artisan migrate --seed
- Executes php artisan test

5. Pass/Fail Criteria

- All automated tests must pass (0 failures)
- All manual acceptance tests must produce expected results
- No JavaScript console errors on any page load
- All API endpoints return correct HTTP status codes
- Role-based UI elements render correctly for each role

6. Known Limitations

- Unit tests use SQLite in-memory; production uses MySQL — some MySQL-specific features are not tested in CI
- Frontend JavaScript modules are not covered by automated tests; manual testing is required
- Performance testing (load testing) is out of scope for this version

